



Integridad de la base de datos, transacciones y concurrencia

Una exploración exhaustiva de cómo los sistemas de bases de datos modernos mantienen la coherencia y la fiabilidad cuando varios usuarios acceden y modifican los datos simultáneamente.

Agenda del curso

1

Fundamentos de la integridad

Comprender los principios de integridad, consistencia y disponibilidad de los datos en los sistemas de bases de datos

2

Transacciones ACID

Análisis de las cuatro propiedades críticas que garantizan un procesamiento fiable de las transacciones

3

Desafíos de la concurrencia

Identificación de problemas comunes que surgen cuando varios usuarios acceden a los datos simultáneamente

4

Técnicas de control

Exploración de métodos prácticos para gestionar el acceso simultáneo manteniendo la coherencia de los datos

5

Aplicación práctica

Implementación de la gestión de transacciones y el control de la concurrencia en entornos de bases de datos reales

Objetivos de aprendizaje



- Comprender los principios de **integridad, consistencia y disponibilidad** en los sistemas de bases de datos
- Analizar las **propiedades ACID** necesarias para el procesamiento fiable de transacciones
- Explorar métodos para mantener la coherencia de los datos durante el **acceso concurrente**
- Identificar y aplicar **técnicas de control de concurrencia** para evitar anomalías en los datos

Por qué esto es importante

Aplicaciones en el mundo real

Los sistemas bancarios, las plataformas de comercio electrónico, las bases de datos de atención médica y los sistemas de reservas de aerolíneas dependen de una gestión de transacciones y un control de concurrencia adecuados para funcionar de manera confiable.

Fiabilidad del sistema

Sin los controles de integridad adecuados, las bases de datos pueden volverse rápidamente poco confiables, lo que provocaría corrupción de datos, pérdidas financieras y fallas críticas del sistema.

Requisito profesional

Comprender estos conceptos es esencial para los administradores de bases de datos, los ingenieros de software y los arquitectos de sistemas que trabajan con cualquier aplicación de base de datos multiusuario.

Integridad, coherencia y disponibilidad

Definiciones clave

- **Integridad de los datos:** Precisión, integridad y confiabilidad de los datos durante todo su ciclo de vida
- **Coherencia:** Estado de la base de datos que satisface todas las reglas y restricciones definidas
- **Disponibilidad:** Medida del tiempo de actividad y la accesibilidad del sistema para los usuarios

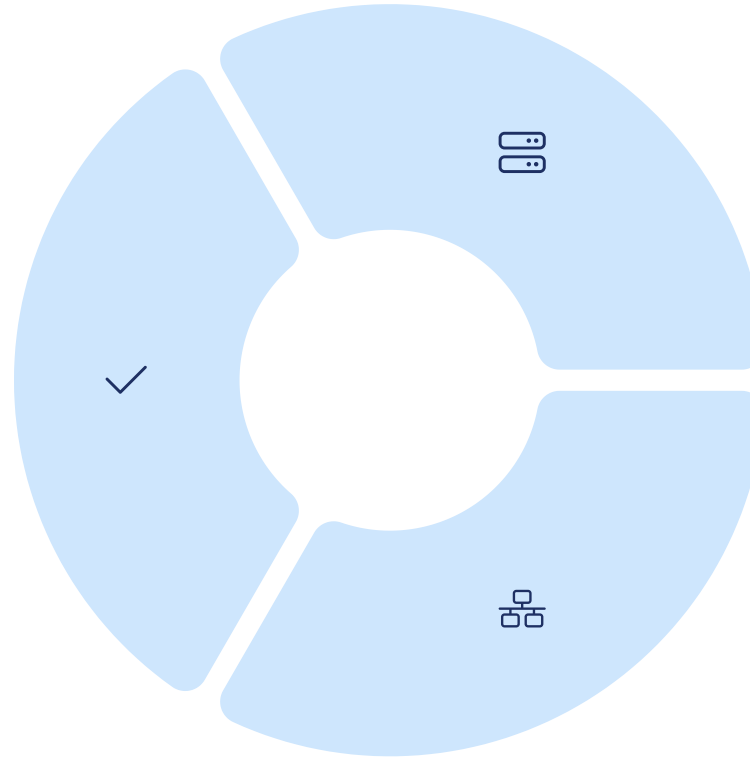
Juntos, estos forman la base de las operaciones confiables de la base de datos, especialmente en entornos multiusuario.



El teorema CAP

Consistencia

Todos los nodos ven los mismos datos al mismo tiempo. Cada lectura recibe la escritura más reciente o un error.



Disponibilidad

Cada solicitud recibe una respuesta, sin garantía de que contenga la versión más reciente de la información.

Tolerancia a particiones

El sistema continúa operando a pesar de las particiones de red o las interrupciones de la comunicación entre los nodos.

Según el teorema CAP, un sistema de base de datos distribuida solo puede garantizar dos de estas tres propiedades simultáneamente. Esta compensación fundamental da forma a las decisiones de arquitectura de la base de datos.

Violaciones de integridad: Ejemplos

Problemas comunes de integridad

- Una cuenta bancaria que muestra diferentes saldos en diferentes vistas del sistema
- Recuentos de inventario que no coinciden con el stock real
- Relaciones de clave externa que apuntan a registros inexistentes
- Claves primarias duplicadas en conjuntos de datos combinados
- Estados de datos contradictorios después de actualizaciones parciales



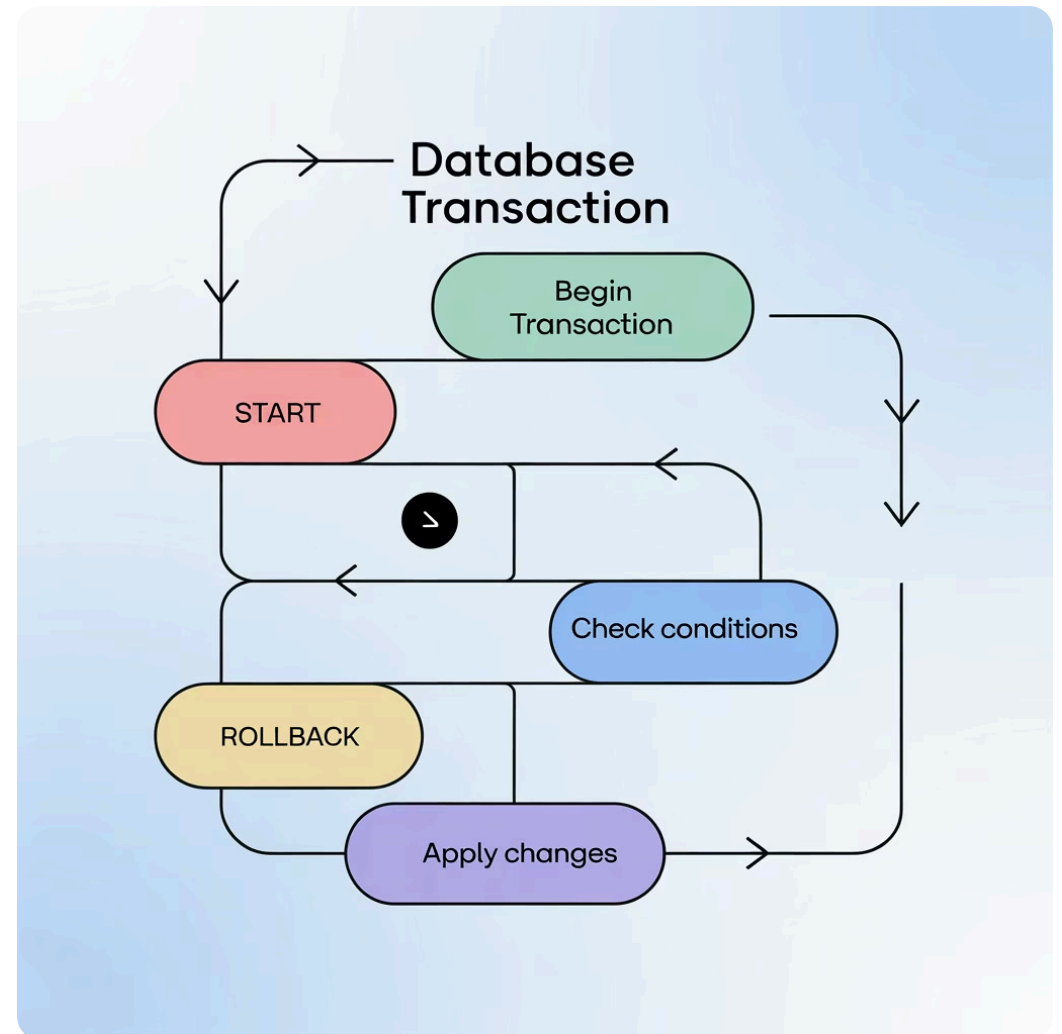
Las violaciones de integridad pueden propagarse en cascada a través de los sistemas, causando problemas de corrupción de datos cada vez más graves si no se gestionan adecuadamente.

¿Qué es una transacción?

Una transacción es una unidad lógica de trabajo que debe completarse en su totalidad para mantener la integridad de la base de datos.

Características clave

- Representa un proceso de negocio completo
- Puede involucrar múltiples operaciones de base de datos
- Debe dejar la base de datos en un estado consistente
- Es la unidad básica de recuperación en caso de falla



El ejemplo clásico: Transferencia bancaria



Paso 1

Verificar si la Cuenta A tiene fondos suficientes



Paso 2

Debitar \$500 de la Cuenta A



Paso 3

Acreditar \$500 a la Cuenta B



Paso 4

Registrar la transacción en el registro

Si algún paso falla, toda la transacción debe revertirse para mantener la coherencia. El dinero no puede desaparecer ni crearse durante las transferencias.

Estados de transacción

Activo

El estado inicial; la transacción ejecuta sus operaciones de lectura/escritura

Parcialmente comprometido

Después de que se ejecuta la operación final, pero antes de que la transacción se confirme en el disco

Comprometido

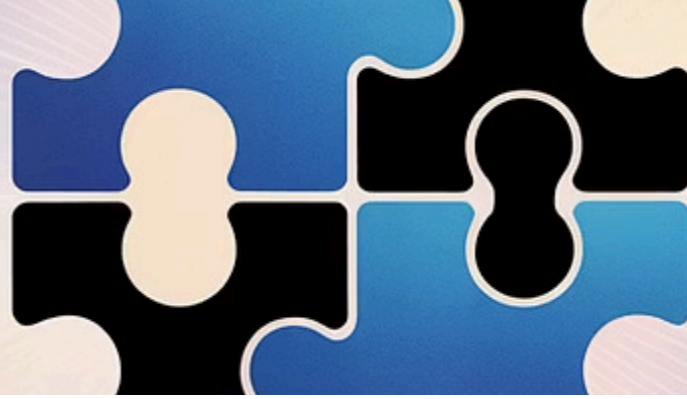
Después de la finalización exitosa; los cambios se almacenan permanentemente

Fallido

Después de determinar que la ejecución normal ya no puede continuar

Anulado

Después de que la transacción se ha revertido y la base de datos se ha restaurado



Propiedades ACID

La base de las transacciones confiables

ACID es un acrónimo que representa las cuatro propiedades críticas que garantizan el procesamiento confiable de transacciones en los sistemas de bases de datos, incluso en presencia de errores, fallas de energía y otros percances.

Atomicidad: Todo o nada

Definición

Una transacción se trata como una unidad única e indivisible que se completa por completo o falla por completo, sin ejecución parcial.

Mecanismos de implementación

- Registro de escritura anticipada (WAL)
- Paginación sombra
- Registro de reversión



- ❗ Si se produce un fallo de alimentación durante una transacción, el proceso de recuperación de la base de datos utilizará los registros de transacciones para completar o deshacer las transacciones parciales.

Consistencia: Mantenimiento de las reglas

Definición

Una transacción lleva la base de datos de un estado válido a otro, asegurando que se mantengan todas las restricciones de integridad de los datos.

Lo que se debe preservar

- Integridad de la entidad (restricciones de clave primaria)
- Integridad referencial (restricciones de clave externa)
- Restricciones de dominio (reglas de tipo de datos)
- Reglas de negocio definidas por el usuario



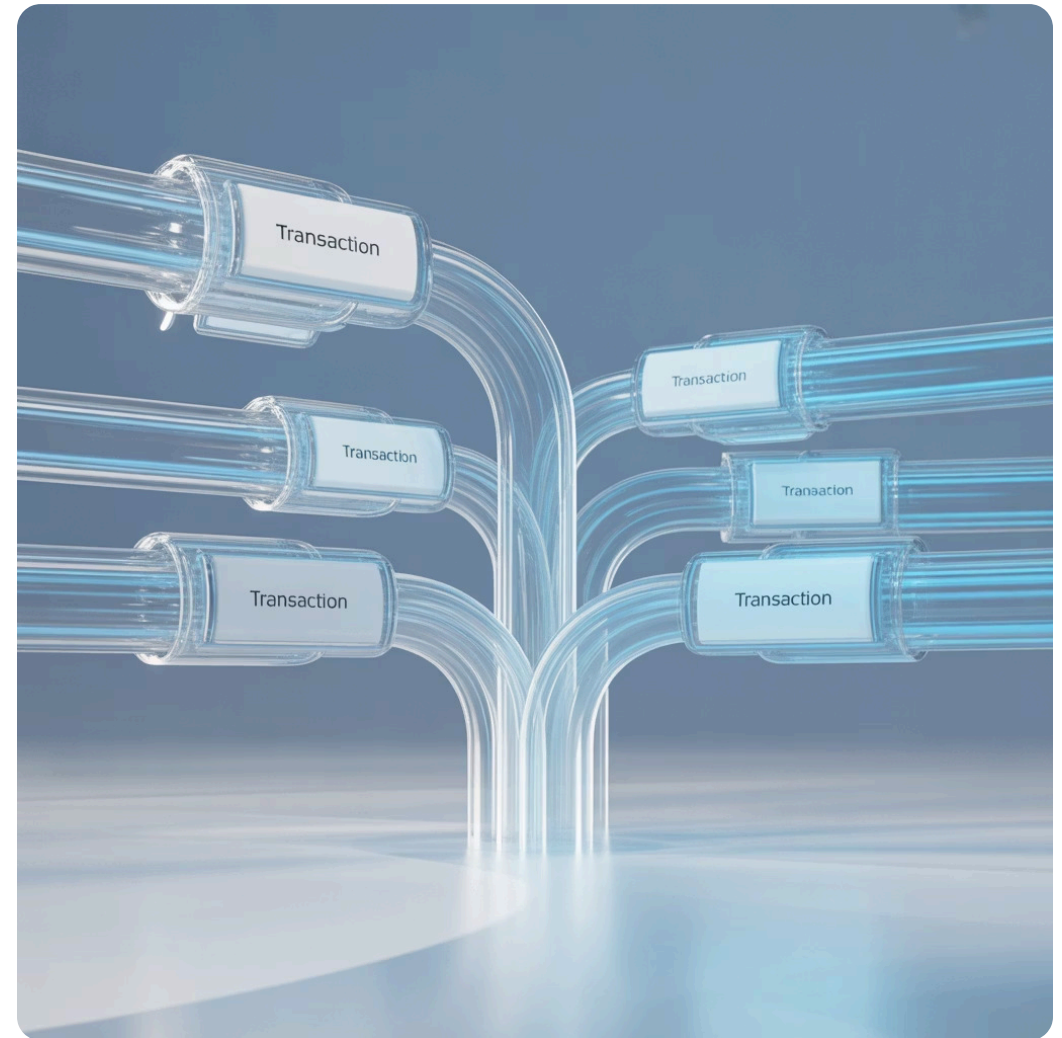
La consistencia garantiza que, sin importar cuándo acceda a la base de datos, la encontrará en un estado que tenga sentido lógico de acuerdo con sus reglas definidas.

Aislamiento: Prevención de interferencias

El aislamiento garantiza que las transacciones simultáneas se ejecuten como si se estuvieran ejecutando secuencialmente, evitando que interfieran entre sí.

Sin aislamiento

- La transacción A podría ver resultados parciales de la transacción B
- Las actualizaciones podrían sobrescribirse sin saberlo
- Los resultados serían impredecibles e inconsistentes



Niveles de aislamiento

Los diferentes niveles de aislamiento ofrecen ventajas y desventajas entre el rendimiento y la coherencia:

- Lectura no confirmada
- Lectura confirmada
- Lectura repetible
- Serializable

Durabilidad: Persistencia permanente

Definición

Una vez que se ha confirmado una transacción, sus cambios persistirán incluso en caso de una falla posterior del sistema.

Mecanismos de implementación

- Registros de transacciones escritos en almacenamiento no volátil
- Puntos de control de la base de datos
- Sistemas de almacenamiento RAID
- Procedimientos de copia de seguridad y recuperación



- ❑ Sin durabilidad, una falla de energía inmediatamente después de una confirmación podría causar la pérdida de transacciones confirmadas.

Control de transacciones SQL

Comandos básicos

```
-- Iniciar una nueva transacción  
START TRANSACTION;
```

```
-- Realizar cambios  
UPDATE accounts  
SET balance = balance - 500  
WHERE account_id = 123;
```

```
UPDATE accounts  
SET balance = balance + 500  
WHERE account_id = 456;
```

```
-- Confirmar cambios  
COMMIT;
```

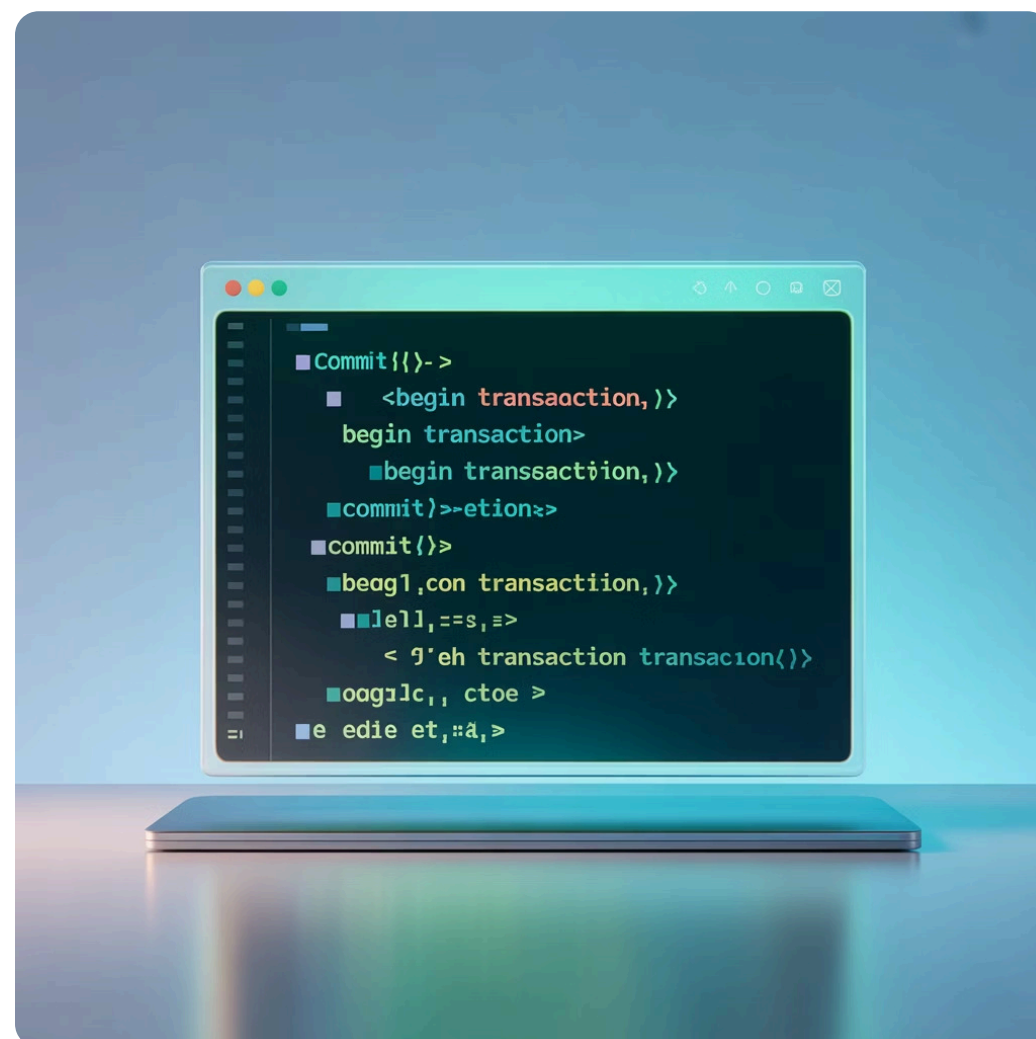
```
-- O descartar cambios  
ROLLBACK;
```

Controles adicionales

```
-- Crear un punto de guardado  
SAVEPOINT transfer_point;
```

```
-- Revertir al punto de guardado  
ROLLBACK TO transfer_point;
```

```
-- Establecer el nivel de aislamiento  
SET TRANSACTION ISOLATION LEVEL  
READ COMMITTED;
```



Desafíos de la concurrencia

Cuando varios usuarios acceden y modifican los mismos datos simultáneamente, pueden surgir varios tipos de problemas si no se implementa un control de concurrencia adecuado:



Problema de actualización perdida

Ocurre cuando dos transacciones leen y luego actualizan los mismos datos, y la segunda transacción sobrescribe los cambios de la primera transacción sin incorporarlos.



Problema de lectura sucia

Ocurre cuando una transacción lee datos que han sido modificados por otra transacción que aún no se ha confirmado, lo que podría dar lugar a la lectura de datos que se revertirán.



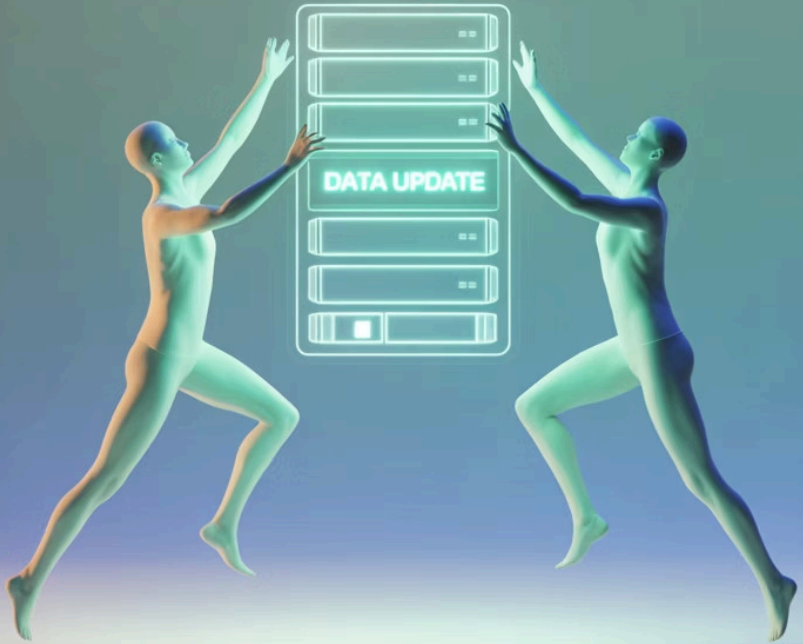
Lectura no repetible

Ocurre cuando una transacción lee los mismos datos varias veces, pero obtiene valores diferentes porque otra transacción ha modificado los datos entre lecturas.



Lectura fantasma

Ocurre cuando una transacción vuelve a ejecutar una consulta que devuelve un conjunto de filas, pero el conjunto ha cambiado porque otra transacción ha insertado o eliminado filas.



El problema de la actualización perdida

Escenario

1. La transacción A lee un registro (inventario = 100)
2. La transacción B lee el mismo registro (inventario = 100)
3. La transacción A calcula el nuevo valor (inventario = 90)
4. La transacción B calcula el nuevo valor (inventario = 95)
5. La transacción A escribe su actualización (inventario = 90)
6. La transacción B escribe su actualización, sobrescribiendo A (inventario = 95)

Consecuencias

La actualización de la transacción A se pierde por completo. Si se tratara de deducciones de inventario, se eliminarían 10 artículos del inventario en el sistema, pero solo se deducirían 5 del registro de la base de datos.

Prevención

- Mecanismos de bloqueo
- Control de concurrencia optimista
- Técnicas basadas en versiones

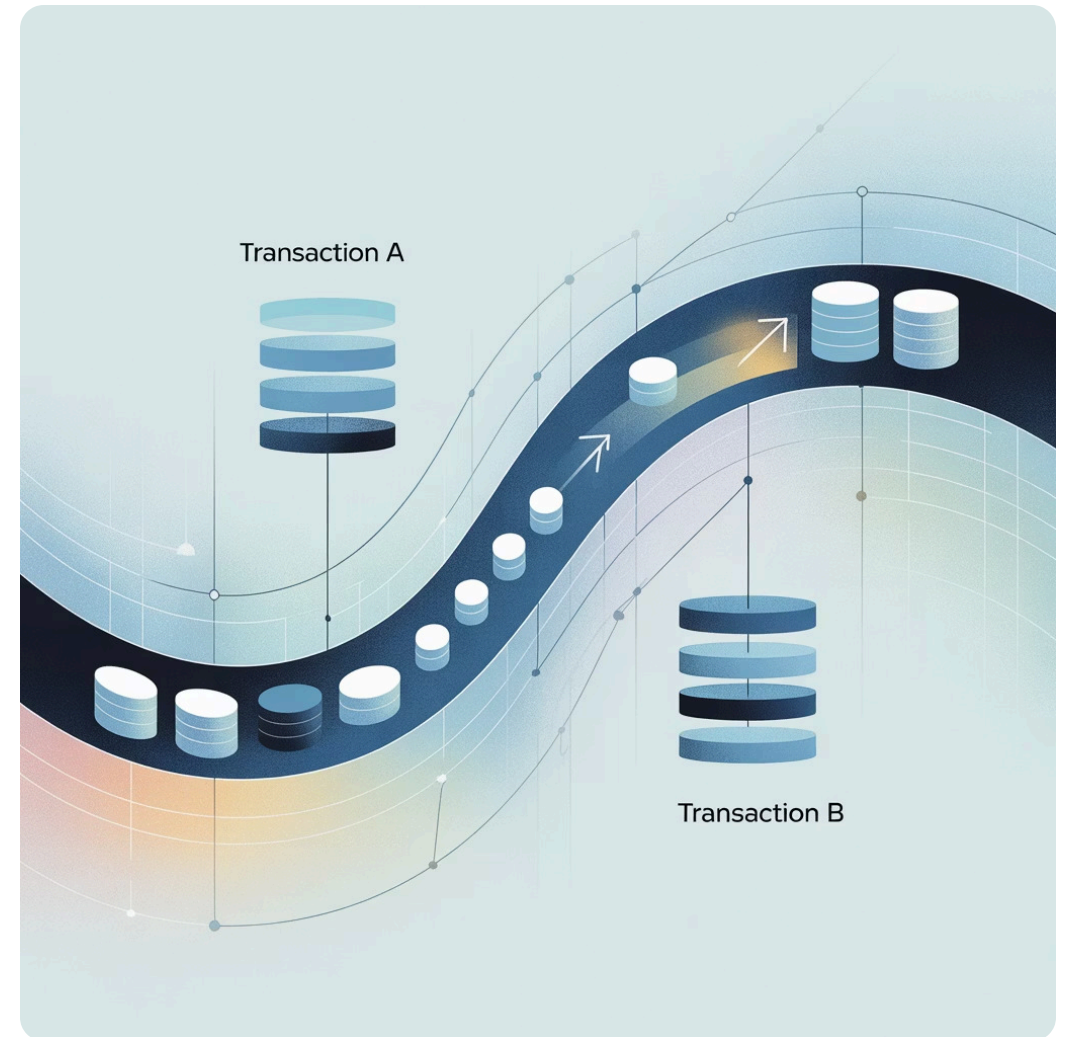
El problema de la lectura sucia

Escenario

1. La transacción A actualiza un registro (saldo = \$1000)
2. La transacción B lee el registro actualizado (ve \$1000)
3. La transacción A encuentra un error y revierte (el saldo vuelve a \$800)
4. La transacción B continúa procesando con datos no válidos (\$1000)

Consecuencias

La transacción B está trabajando con datos que en realidad nunca existieron en un estado confirmado, lo que podría provocar violaciones de las reglas de negocio o errores en cascada.



Prevención

- Establezca el nivel de aislamiento al menos en READ COMMITTED
- Use el control de versiones de filas
- Implemente bloqueos de lectura

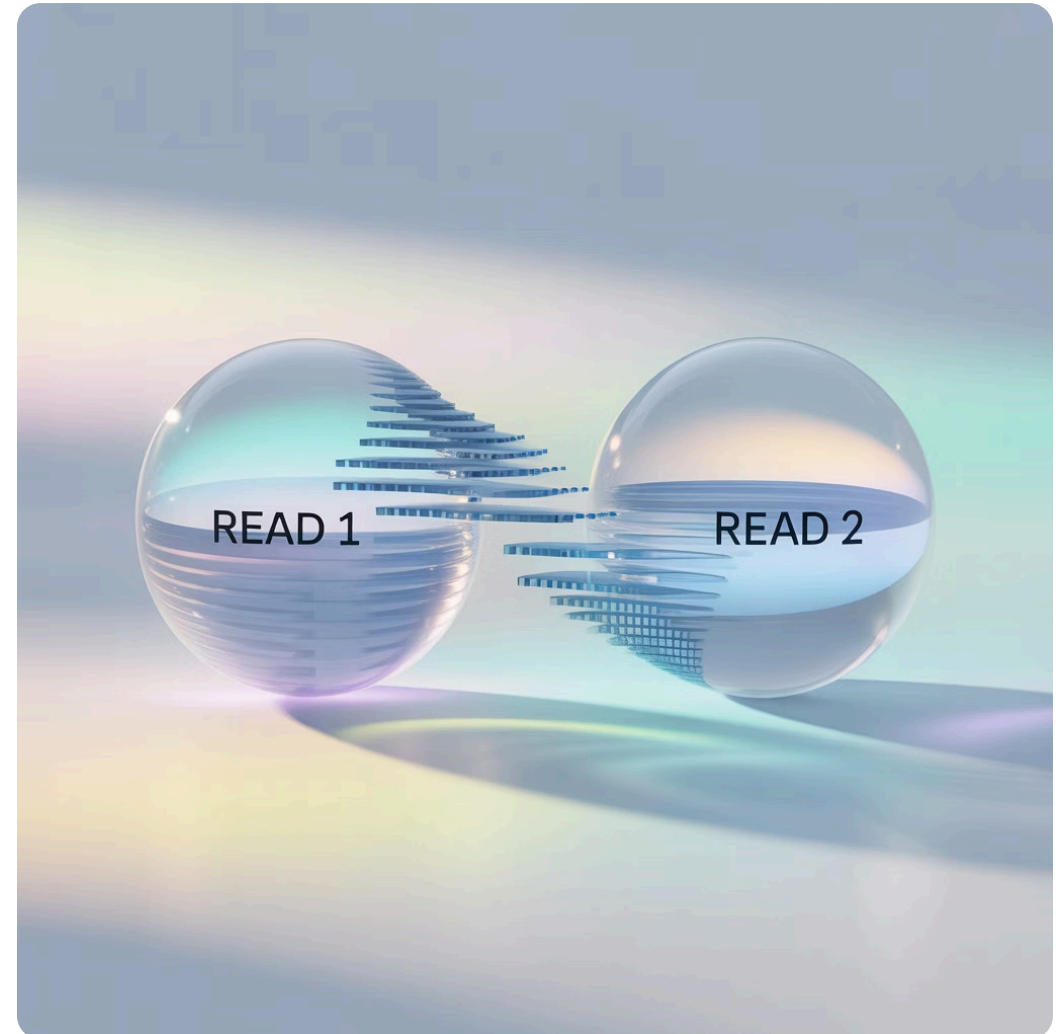
Problema de lectura no repetible

Escenario

1. La transacción A lee un registro (precio = \$10.00)
2. La transacción B actualiza y confirma el mismo registro (precio = \$12.00)
3. La transacción A vuelve a leer el registro (ahora ve \$12.00)
4. La lógica empresarial de la transacción A falla debido a datos inconsistentes

Consecuencias

La transacción A no puede confiar en que los datos se mantengan consistentes durante su ejecución, lo que dificulta la aplicación de reglas empresariales que dependen de valores estables.



Prevención

- Establezca el nivel de aislamiento en al menos LECTURA REPETIBLE
- Implemente bloqueos de lectura que persistan hasta la finalización de la transacción
- Utilice el aislamiento de instantáneas en las bases de datos que lo admitan

Problema de Lectura Fantasma

Escenario

1. La transacción A consulta los registros que coinciden con los criterios (encuentra 5 registros)
2. La transacción B inserta un nuevo registro que coincide con los criterios
3. La transacción A repite la misma consulta (ahora encuentra 6 registros)
4. El procesamiento de la transacción A se ve comprometido por el registro inesperado

Consecuencias

Los cálculos estadísticos, las funciones agregadas y los procesos empresariales que dependen de un conjunto estable de registros pueden producir resultados incorrectos.



Prevención

- Establecer el nivel de aislamiento en `SERIALIZABLE`
- Implementar bloqueos de rango o bloqueos de predicado
- Usar bloqueos de tabla en casos extremos

Niveles de aislamiento

Nivel de aislamiento	Lectura sucia	Lectura no repetible	Lectura fantasma
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	Impedido	Posible	Posible
REPEATABLE READ	Impedido	Impedido	Posible
SERIALIZABLE	Impedido	Impedido	Impedido

Los niveles de aislamiento más altos brindan más consistencia, pero generalmente reducen la concurrencia y pueden afectar el rendimiento. El nivel adecuado depende de los requisitos de la aplicación.

Técnicas de control de concurrencia



Mecanismos de bloqueo

Restringir el acceso a los elementos de datos para evitar el acceso y la modificación simultáneos



Ordenamiento de marca de tiempo

Asignar marcas de tiempo a las transacciones y procesar las operaciones según el orden de las marcas de tiempo



Control de concurrencia multiversión

Mantener varias versiones de elementos de datos para permitir lecturas simultáneas sin bloqueo



Control de concurrencia optimista

Permitir que las transacciones continúen sin bloqueo, pero validar los cambios antes de confirmar

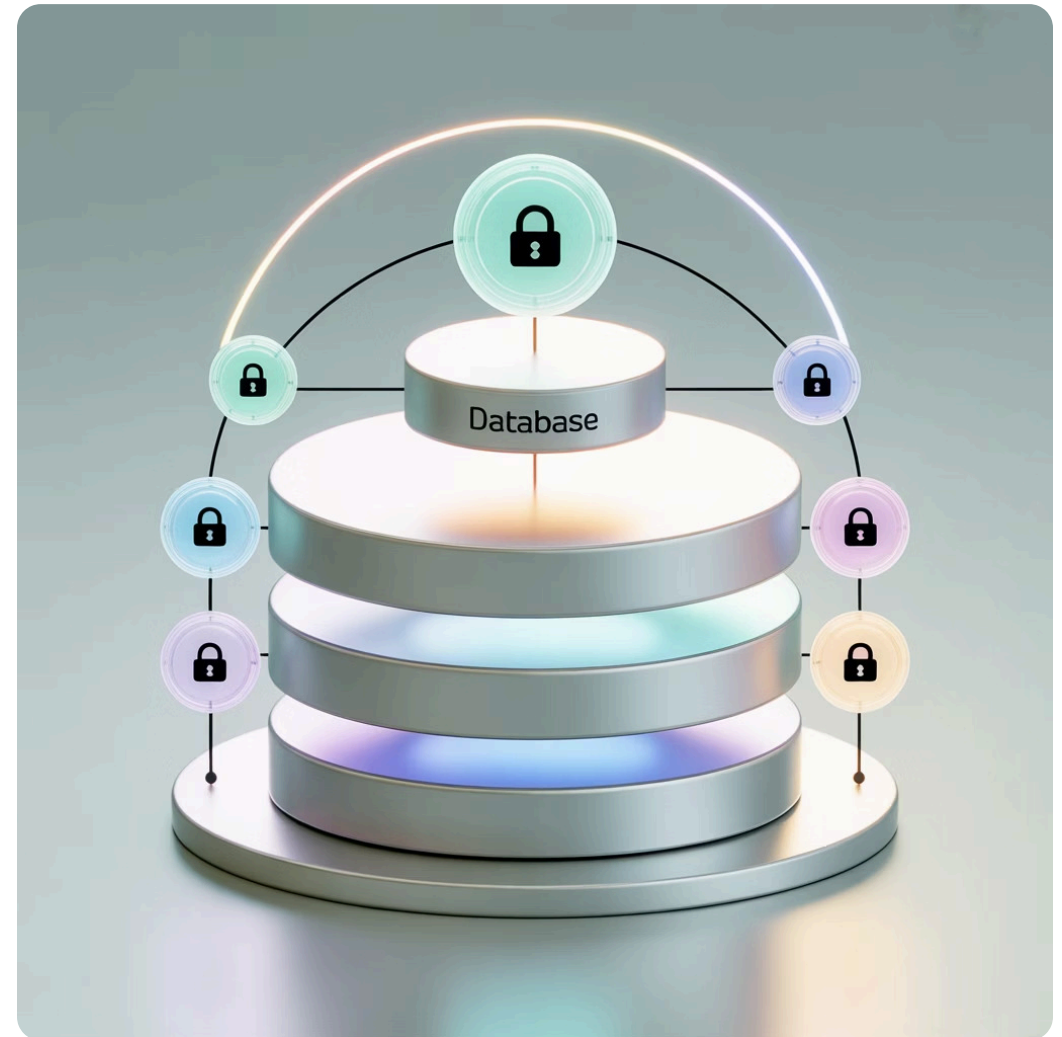
Estas técnicas emplean diferentes estrategias para equilibrar las ventajas y desventajas entre coherencia, rendimiento y disponibilidad.

Mecanismos de bloqueo

Tipos de bloqueo

- **Bloqueos compartidos (S):** Permiten lecturas simultáneas pero impiden escrituras
- **Bloqueos exclusivos (X):** Impiden tanto las lecturas como las escrituras simultáneas
- **Bloqueos de actualización (U):** Permiten lecturas pero señalan la intención de actualización potencial
- **Bloqueos de intención:** Señalan las operaciones planificadas en los recursos subordinados

Granularidad de bloqueo



- **Nivel de base de datos:** Base de datos completa (rara vez se usa)
- **Nivel de tabla:** Tabla completa
- **Nivel de página:** Bloque de almacenamiento físico
- **Nivel de fila:** Registros individuales
- **Nivel de columna:** Atributos específicos (raro)

Bloqueo de Dos Fases (2PL)

Fase de Crecimiento
La transacción puede obtener bloqueos, pero no puede liberar ninguno



Fase de Reducción
La transacción puede liberar bloqueos, pero no puede obtener nuevos

2PL Estricto

Los bloqueos exclusivos se mantienen hasta que la transacción se confirma o se anula, lo que evita las reversiones en cascada si falla una transacción.

2PL Riguroso

Todos los bloqueos (tanto compartidos como exclusivos) se mantienen hasta que la transacción se confirma o se anula, lo que proporciona el nivel más alto de aislamiento.

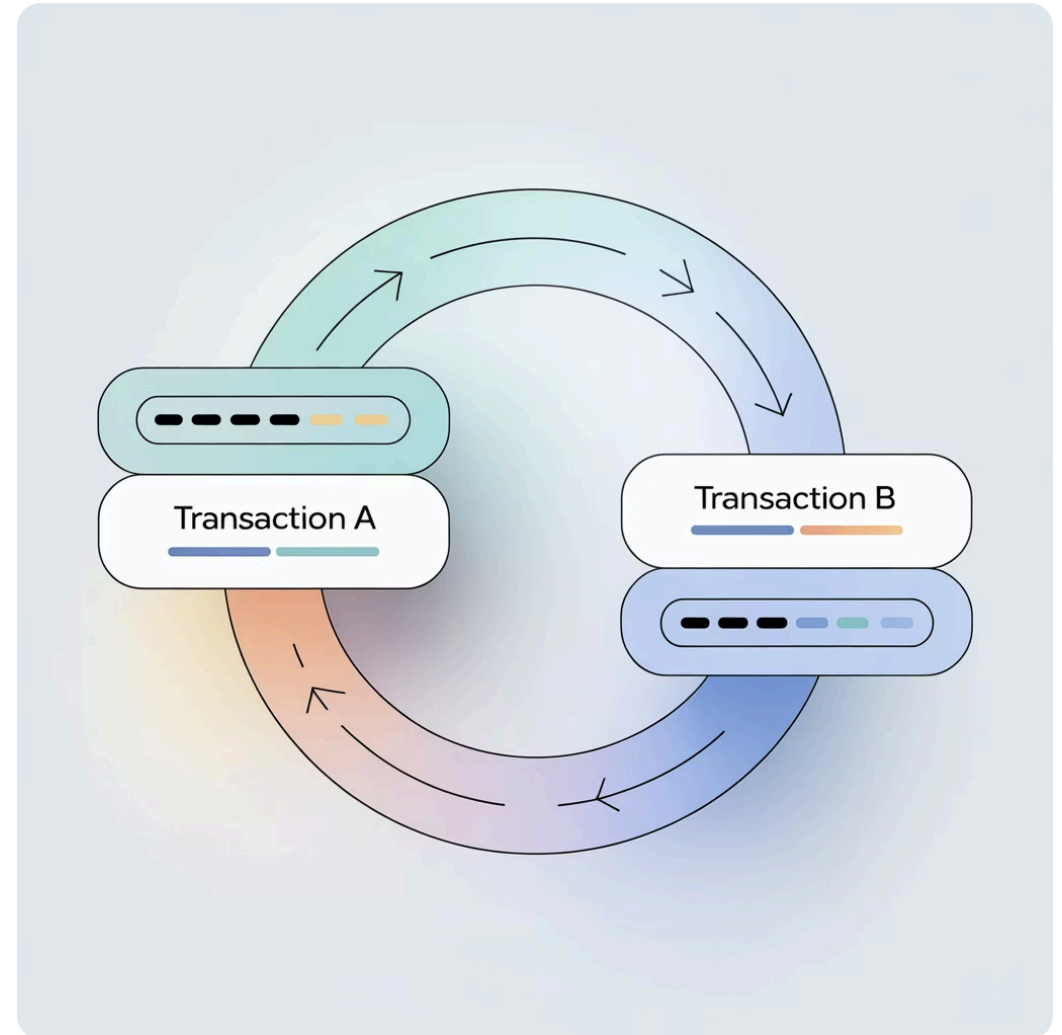
Interbloqueos: El peligro de la concurrencia

¿Qué es un interbloqueo?

Un interbloqueo ocurre cuando dos o más transacciones están esperando indefinidamente a que la otra libere los bloqueos, creando una dependencia circular que impide que cualquiera de ellas avance.

Escenario de ejemplo

1. La transacción A bloquea el recurso 1, necesita el recurso 2
2. La transacción B bloquea el recurso 2, necesita el recurso 1
3. Ambas transacciones esperan indefinidamente



Estrategias de manejo

- **Prevención:** Obligar a las transacciones a adquirir bloqueos en un orden consistente
- **Detección:** Usar gráficos de espera para identificar dependencias circulares
- **Resolución:** Seleccionar una transacción víctima para abortar y reiniciar
- **Tiempos de espera:** Abortar automáticamente las transacciones que esperan demasiado

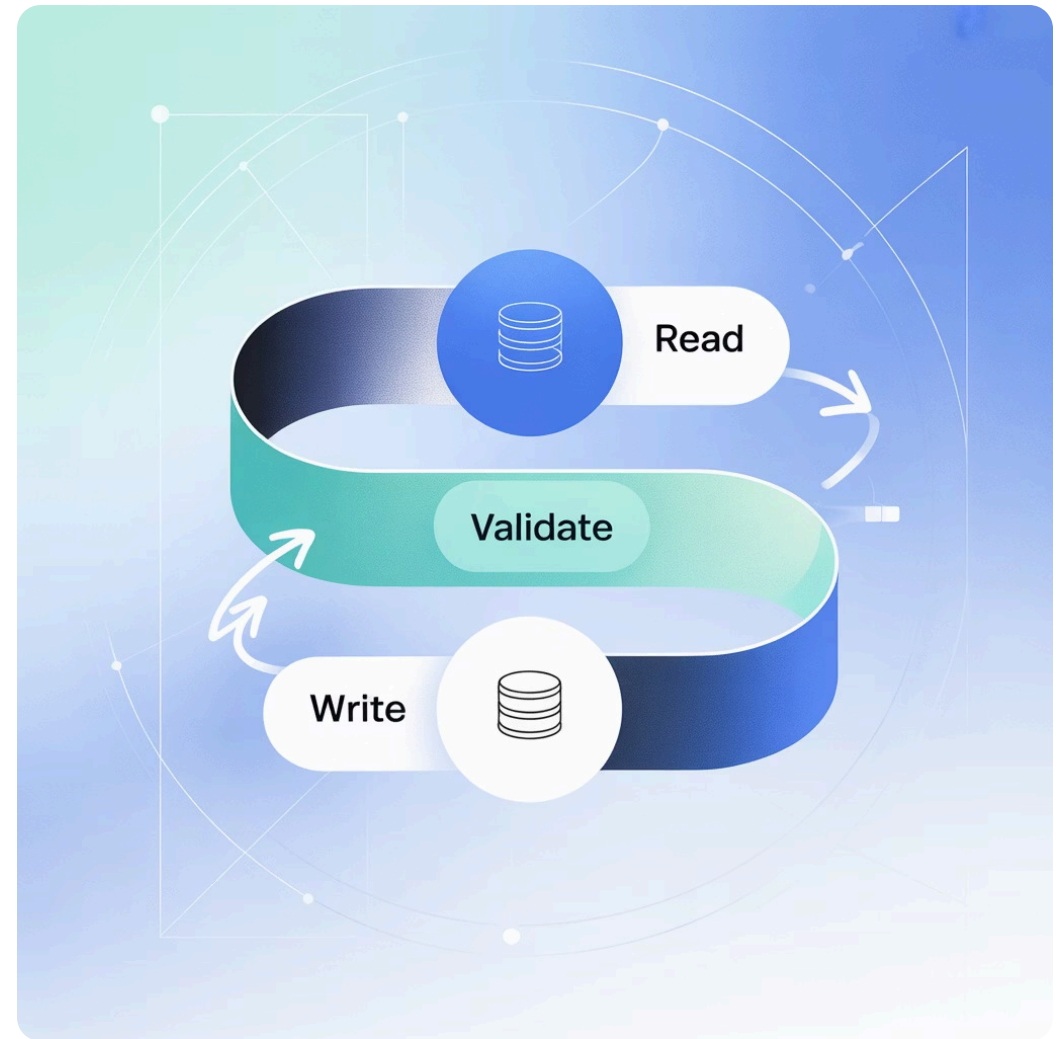
Control de concurrencia optimista

Concepto central

Asume que los conflictos entre transacciones son raros y permite que las transacciones continúen sin bloqueo. Valida los cambios antes de confirmar para garantizar que no se produzcan conflictos.

Tres fases

1. **Lectura:** la transacción lee los datos y guarda copias locales
2. **Validación:** antes de confirmar, verifica que no haya conflictos con otras transacciones
3. **Escritura:** si la validación se realiza correctamente, realiza cambios permanentes



Ventajas

- Sin bloqueo, mayor concurrencia
- Evita los interbloqueos por completo
- Eficiente cuando los conflictos son raros

Desventajas

- Trabajo desperdiciado si las transacciones fallan en la validación
- Lógica compleja de detección de conflictos

- Los lectores nunca bloquean a los escritores
- Los escritores nunca bloquean a los lectores
- Excelente para cargas de trabajo con muchas lecturas
- Admite el aislamiento de instantáneas

Ejercicio práctico: Transacción de transferencia bancaria

Instrucciones del ejercicio

1. Cree una tabla de cuentas con ID, nombre y saldo
2. Escriba un script de transacción para transferir dinero entre cuentas
3. Agregue el manejo de errores para evitar saldos negativos
4. Pruebe con acceso concurrente desde múltiples sesiones de phpMyAdmin
5. Observe los efectos de los diferentes niveles de aislamiento

-- Crear tabla de cuentas

```
CREATE TABLE accounts (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  balance DECIMAL(10,2) NOT NULL  
);
```

-- Transacción de transferencia

```
START TRANSACTION;
```

-- Comprobar fondos suficientes

```
SELECT balance FROM accounts  
WHERE id = 101 FOR UPDATE;
```

-- Realizar transferencia si es suficiente

```
UPDATE accounts SET balance = balance - 500  
WHERE id = 101;
```

```
UPDATE accounts SET balance = balance + 500  
WHERE id = 102;
```

-- Comprobar si hay saldo negativo

```
IF (SELECT balance FROM accounts WHERE id = 101) < 0 THEN  
  ROLLBACK;  
ELSE  
  COMMIT;  
END IF;
```



Conclusiones clave

Garantías transaccionales

Las propiedades ACID (atomicidad, consistencia, aislamiento, durabilidad) proporcionan la base para operaciones de bases de datos confiables, lo que garantiza la integridad de los datos incluso durante fallas.

Problemas de concurrencia

Sin los controles adecuados, el acceso concurrente a la base de datos puede provocar la pérdida de actualizaciones, lecturas sucias, lecturas no repetibles y lecturas fantasma que comprometen la integridad de los datos.

Mecanismos de control

El bloqueo, el ordenamiento de marcas de tiempo, el control de concurrencia multiversión y las técnicas optimistas proporcionan diferentes enfoques para administrar el acceso concurrente con diferentes ventajas y desventajas.

Aplicación práctica

Comprender los niveles de aislamiento y la administración de transacciones es esencial para diseñar aplicaciones de bases de datos sólidas que mantengan la coherencia de los datos en entornos multiusuario.